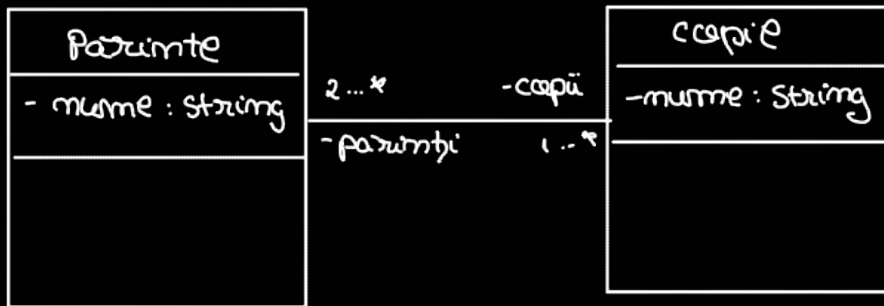


A23

1. Alegerea de a utiliza un nivel dedicat pentru gestiunea datelor persistente ajută sistemul să fie modular, flexibil, aceste lucruri corespunzând cerințelor de întrebare : extensibilitate , modificabilitate și de cost , reducând costul de întrebare .

2. Diagrama de clase care să reprez. rel. dintre părinți și copii .

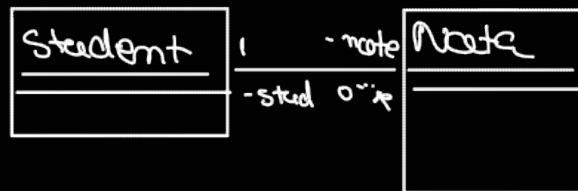


3. Proprietățile unei relații de asociere între clase sunt :

- numele de rol → acestea reprezentând denumirea atributelor prin care obiectul din capatul capus are referința către celălalt

- ordinele de multiplicitate → indică multiplicitate asocierii

(ex: un student poate obține mai multe note, dar o notă poate fi asociată unui singur student :



) → în cod se poate utiliza ca referință simplă sau ca colecție de referințe

- numele relației

- navigabilitatea → determină sensul în care poate fi interogată relația ; în cod , se reprezintă prin care dintre clase ține minte referința către cealaltă

C3

1. Recapitulare principii SOLID

S → Single Responsibility Principle → fiecare clasă are o singură resp.

O → Open closed Principle → deschis la extindere, închis la modificare

L → Liskov Substitution Principle → cu întotdeauna pot să

Încercăm să creștem de la o clasă cu un subtip al său

I → Interfața Segregatam Principale (Principiul separării interfețelor)
cienții nu trebuie constrânși să depindă de interfețe pe care nu le utilizează

→ se bazează în mod special pe separarea unei interfețe mari în interfețe mai mici, pt. a putea porționa clasele să-și asigure ce interfețe vor implementa

b → Dependency Inversion Principle (Principiul inversării dependențelor)

- modulele de nivel înalt nu ar trebui să depindă de module de nivel jos ⇒ ambele trebuie să depindă de abstracții

- abstracțiile nu trebuie să depindă de detalii, ci detaliile de abstracții

Principiul solid al separării interfețelor se bazează pe ideea că micșun client nu ar trebui forțat să depindă de interfețe pe care nu le folosește. Mai exact, în cazul în care avem o interfață ce conține mai multe metode, am prefera să o separăm în interfețe mai mici, a.î. clasele să implementeze flux metodele de care au nevoie.

Spre exemplu, să zicem că avem 3 tipuri de mașini:

- clasică (folosește carburant)
- electrică
- hibridă

În loc să avem o interfață cu metodele `fill-with-gas()` și `charge()`, ar fi mai bine dacă le-am separa în 2 interfețe diferite: `FillGas` și `Charge`. Astfel, mașina clasică poate implementa interfața `FillGas`, cea electrică pe `Charge`, iar cea hibridă pe amândouă, fiecare clasă fiind constrânsă doar de metode pe care le utilizează.

2. Asocierile calificate sunt un mod de reducere a multiplicității asocierii, prin alegerea unui calificativ (care trebuie să fie unic doar la nivel global).

Exemplu : Spectacol și Bilet cu locuri aferent (cu identificare)



Asocierea calificată se implementează cel mai adesea prin dicționare:

```
public class Spectacol {
    private Map<String, Bilet> locuri;
    public Spectacol() {
        locuri = new HashMap();
    }

    public void addBilet (String locBilet, Bilet a) {
        if (!locuri.containsKey (locBilet)) {
            locuri.put (locBilet, a);
            a.internalSetSpectacol(this);
        }
    }

    public Bilet getBilet (String locBilet) {
        return locuri.get (locBilet);
    }

    public void internalAddBilet (String locBilet, Bilet a) {
        if (!locuri.containsKey (locBilet)) {
            locuri.put (locBilet, a);
        }
    }

    public void removeBilet (String locBilet) {
        if (locuri.containsKey (locBilet)) {
            locuri.remove (locBilet);
            a.internalSetSpectacol (null);
        }
    }
}
```

```

public void internalRemoveBilet (String locBilet) {
    if (locBilet.contains key (locBilet))
        locBilet.remove (locBilet);
}
}

```

```

public class Bilet {
    private Spectacol spectacol;
    public Bilet() {}
}

```

```

public void setSpectacol (Spectacol s, String locBilet) {
    Spectacol oldS = this.spectacol;
    if (oldS != null)
        oldS.internalRemoveBilet (locBilet);

    this.spectacol = s;
    s.internalAddBilet (locBilet, this);
}

public void internalSetSpectacol (Spectacol s) {
    this.spectacol = s;
}
}

```



Constrângere: $\hat{I}$ în fiecare proiect, trebuie să existe un singur
 pe el request de cu fiecare contributor.
 context proiect

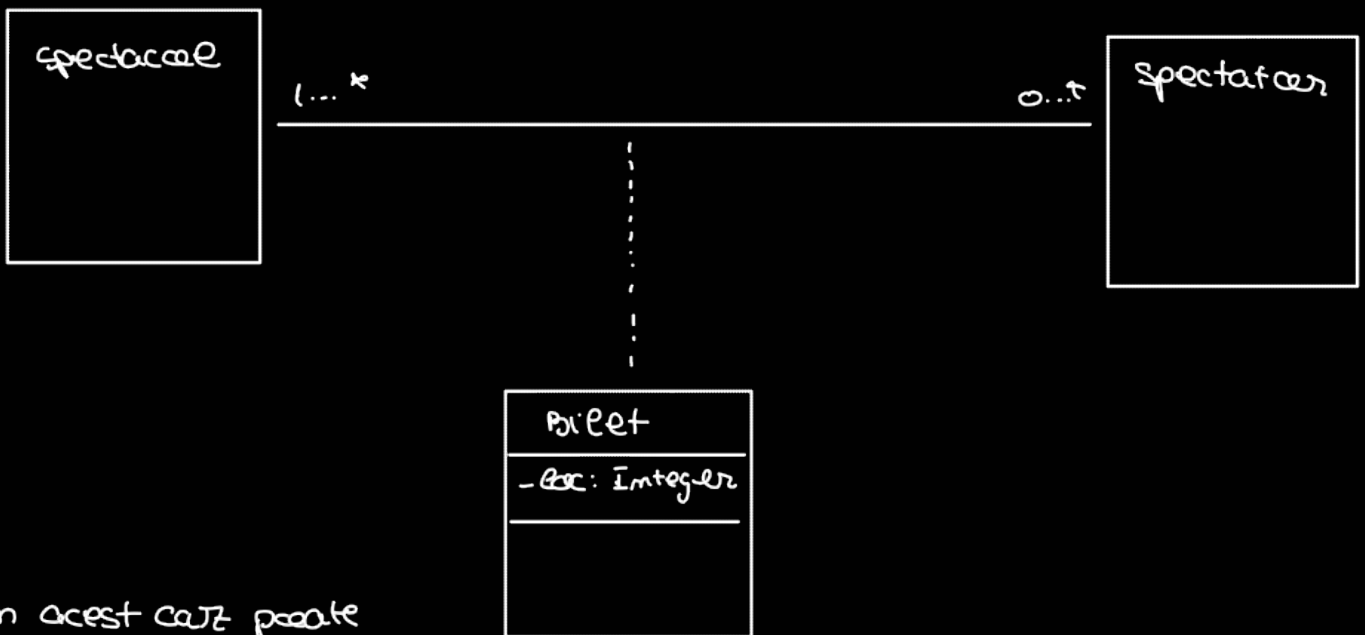
inu Nz Maximum Pz : self. p2z $\rightarrow$ is Unique (c)

A22

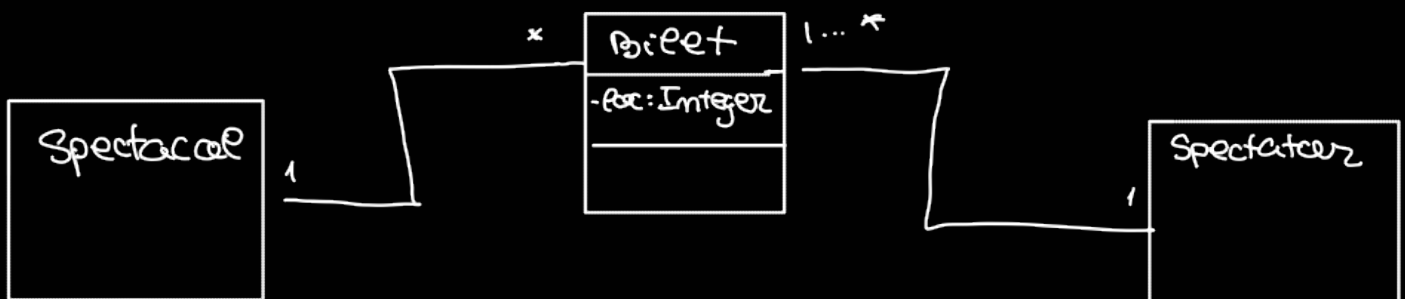
1. Sistemul în cauză nu poate fi considerat un actor, deoarece un actor, prin definiție, este un element extern ce interacționează cu sistemul.

2. În multe cazuri, tabela de legătură (la nivelul bazei de date relaționale) conține și alte informații pe lângă legătura propriu-zisă, care în care descompunerea cu clasă asociere este una necesară (ex: Spectator și Spectacol au nevoie de o tabelă de legătură Bilet, ce conține inclusiv numărul locului rezervat).

Există 2 moduri de reprezentare grafică a clasei asociere (se



- în acest caz poate exista o singură legătură între Spectacol și Spectator



- există posibilitatea de a se realiza legătura dintre Spectacol și

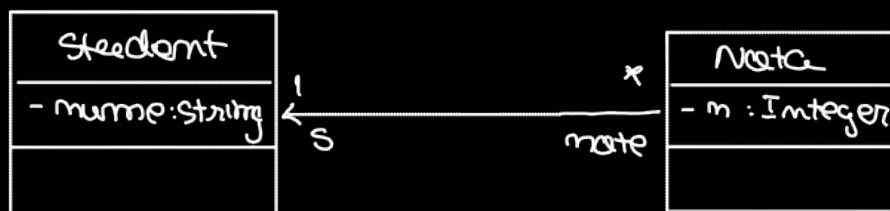
Spectator (dacă locul este dublu → spre exemplu, o singură persoană cumpără bilete pentru un grup de prieteni)

- dacă presupunem că locurile nu sunt rezervate la momentul cumpărării biletului, am putea să folosim asocieri reciprocate (se presupune că numele spectatorilor este unic) în acest caz, din nou spectatorul va putea cumpăra maxim un bilet



3. Numele de rol reprezintă denumirea atributelor folosite de clasă din capătul opus pentru a referenția un obiect din clasă apropiată. La nivelul diagramei clarifică contextul și este util în cazul în care este nevoie să specificăm invariabili, pre, post condiții în CL.

Exemplu: se va considera o asocieră unidirecțională, unde numai clasa Nota ține referință către Student.



public class Student {

private String nume;

public Student (String n) {

nume = n;

}

}

public class Nota {

private Integer m;

private Student s;

public Nota () {};

public Integer getNota ()

return m;

clarifică scopul
asocierii și permit
diferențierea între
asocieri diferite ce
conectează aceleași
clase

```

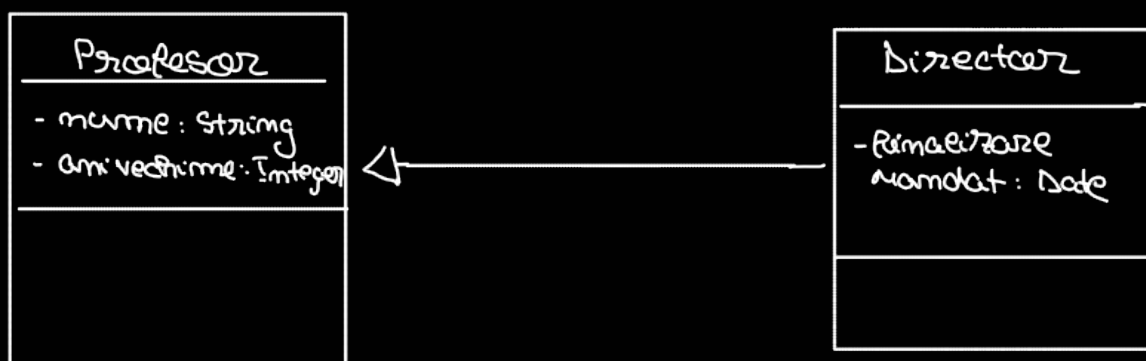
    }
    public void setNota (Integer nota) {
        n = nota;
    }
    public Student getStudent () {
        return s;
    }
    public void setStudent (Student stud) {
        s = stud;
    }
}

```

C2 Design by Contract

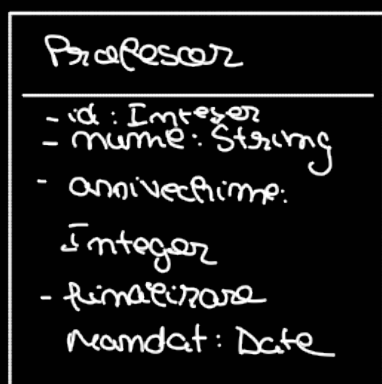
1. În acest caz, responsabilitatea se referă la proprietatea sistemului de a respecta specificațiile, dar și de a putea gestiona situațiile care nu au fost cuprinse în specificații. Mai exact, abordarea Design by Contract este contrară programării defensive, căzându-se pe aserțiuni, recomandări în loc să facă prea multe verificări care, ulterior, duc la complicarea implementării și la posibilitatea apariției unor erori noi.

2. Modalități de reprezentare a ierarhiilor de management



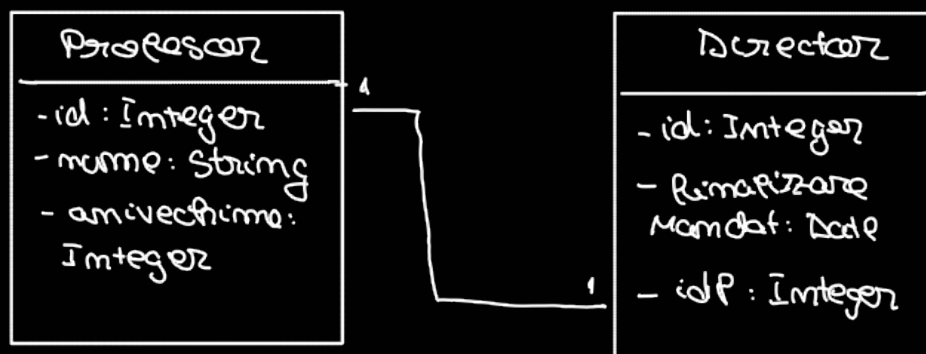
Modalitate 1 : toate într-o singură tabelă

(profesorii care sunt directori se diferențiază prin existența unei date de înregistrare a mandatului)



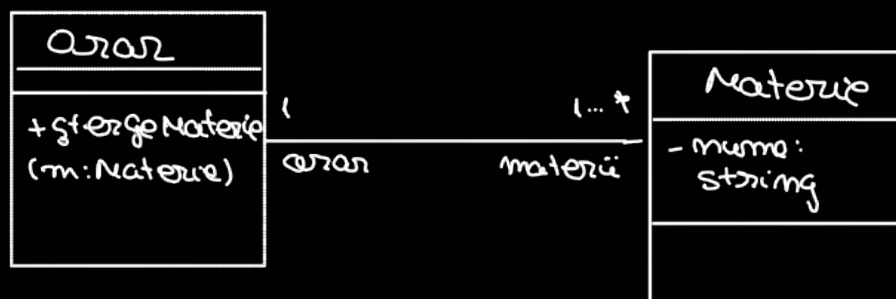
→ pt. profesori care nu sunt directori, finalizare Mandat va avea valoarea null

modalitate a: relatie de 1 la 1



3. Exemple de precondiție care să utilizeze unul dintre iteraorii pe colecții.

Pentru operația de ștergere a unei Materii din Orar, aceasta trebuie să existe în Orar.



Context Orar :: ștergeMaterie (m: Materie)
pre: see. materii → exists (m)

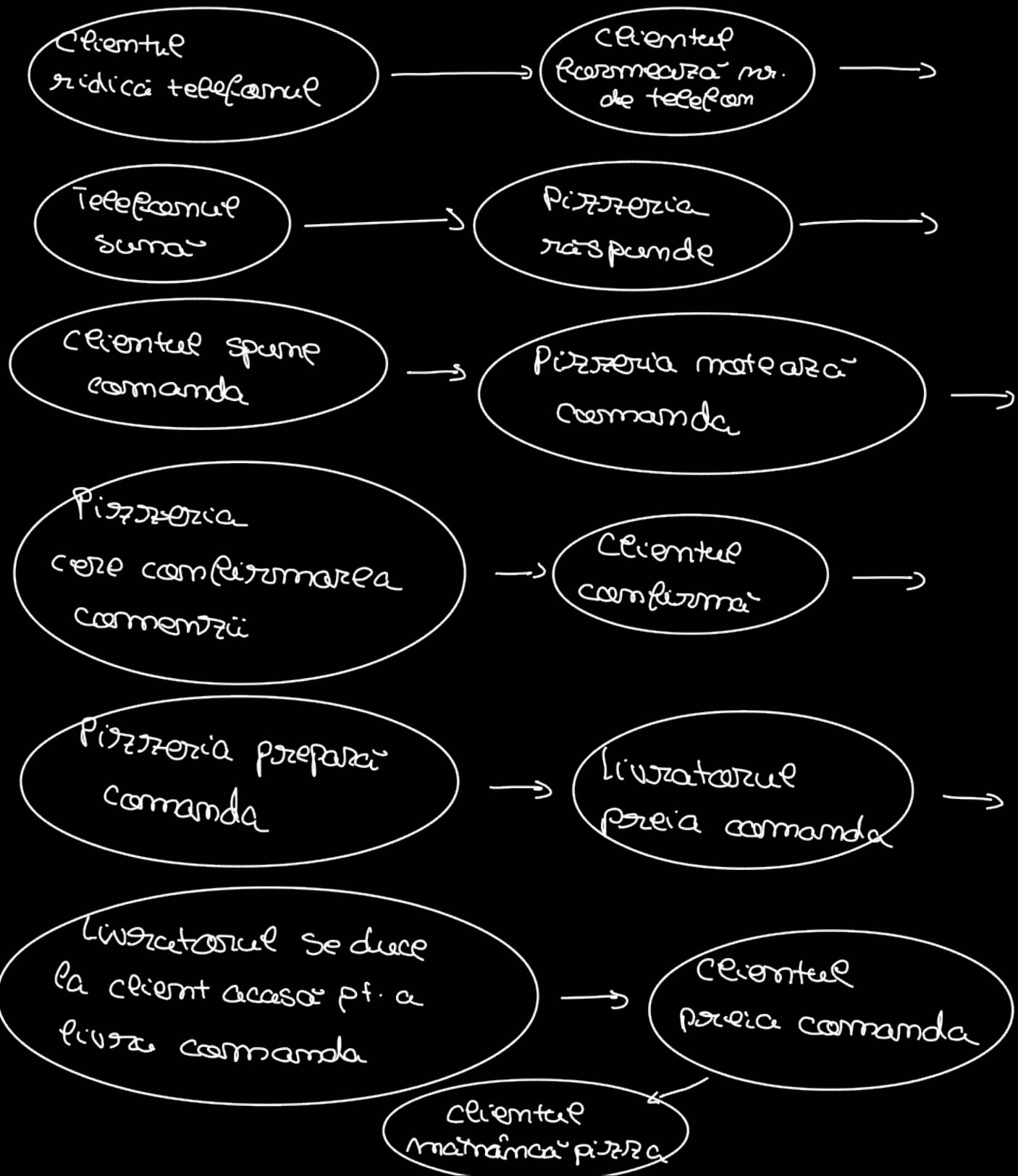
A21

1. În etapa de analiză a cerințelor se structurează modelele dinamice și modelul obiectual de analiză ce stau la baza etapei de proiectare. Mai exact, etapa de analiză

este reprezentativă pentru a înțelege exact care sunt cerințele și cum pot fi modelate, la nivel înalt.

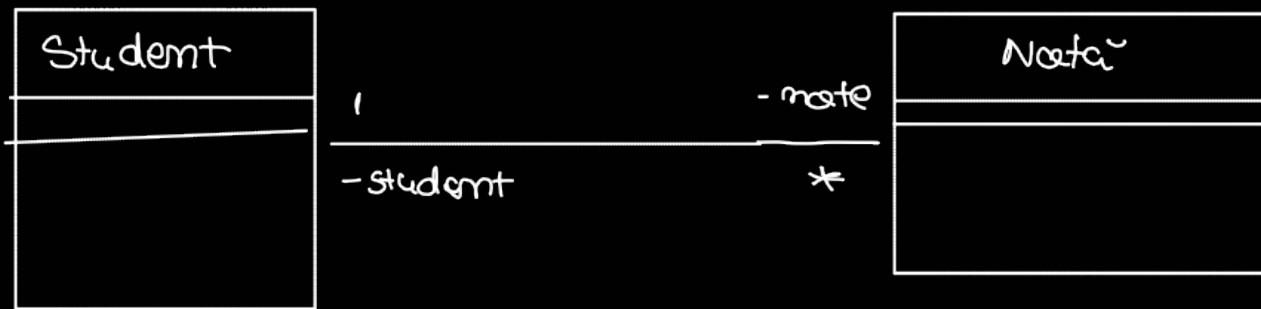
În faza de proiectare se iau în considerare mai multe detalii de implementare, precum arhitectura sistemului și sarcinile de proiectare (pt. proiectarea obiectelor).

2. Diagramă de activități pt. comanda telefonică a unei pizza.



3.

- numele de roluri dau numele rolurilor pe care obiectul clasei din capatul apăs al are spre obiectul clasei apropiate (se pot atribui modificatori de acces)
- multiplicitățile urmează a regula similitudinii; astfel pe baza exemplului de mai jos, un student poate avea mai multe note (inclusiv nici măcar una), dar o notă trebuie să aparțină neapărat unui student



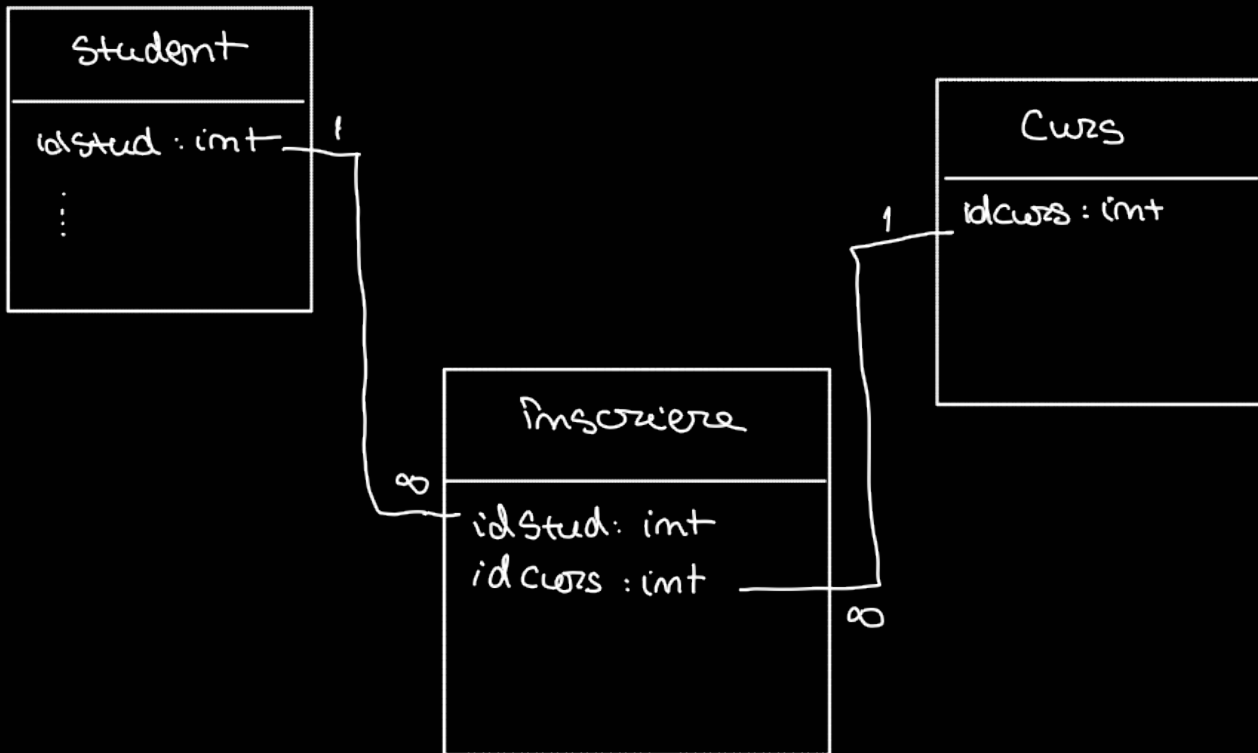
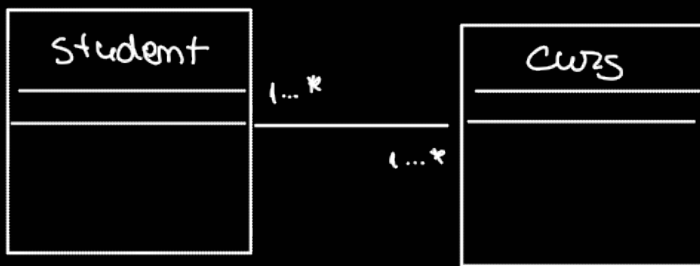
C1

→ entități (clase, module, funcții)

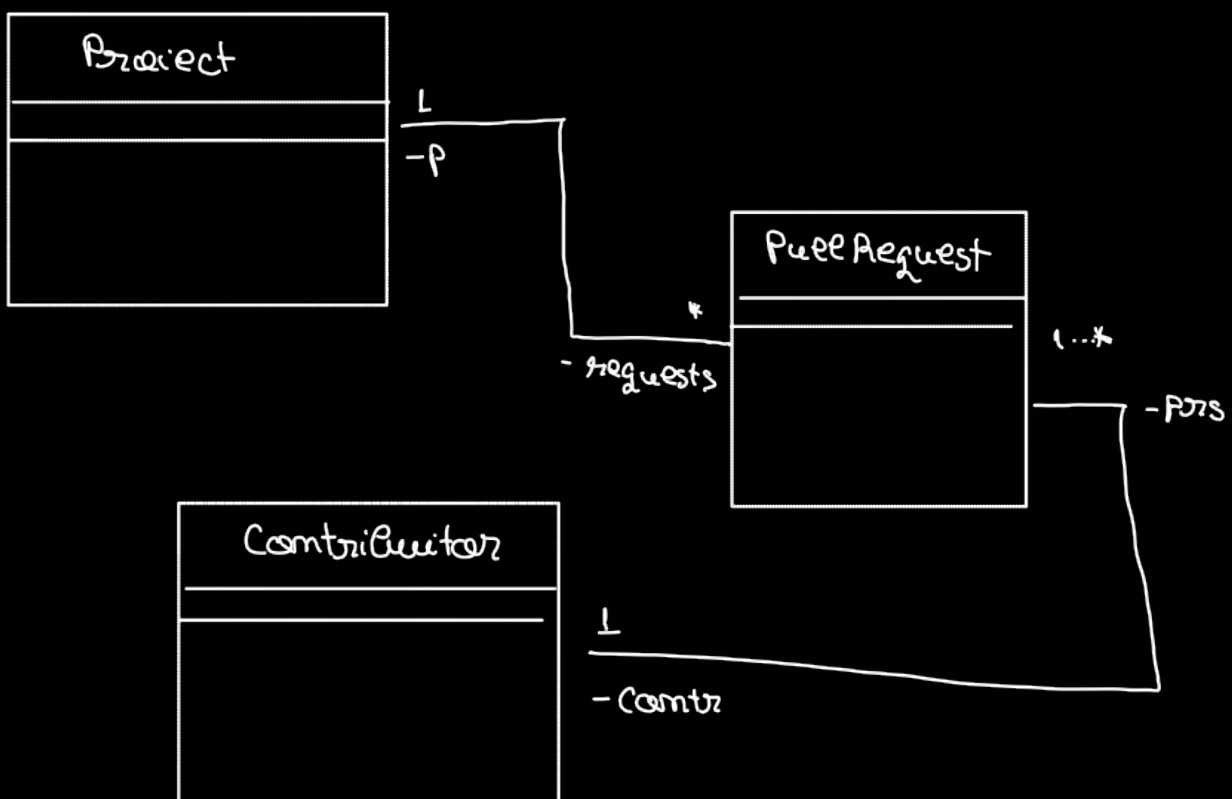
1. Principiul SOLID Open closed se referă la proprietatea unei clase de a fi deschisă la extindere, dar închisă la modificare. Mai exact, un proiect, inevitabil, va trebui extins la un moment dat. Pentru a face acest lucru cât mai puțin costisitor, trebuie gândit ca fiind extensibil. În același timp, nu trebuie să permită modificarea, deoarece acest lucru ar duce la modificarea și la nivelele superioare ale sistemului, având un potențial mare de a introduce erori.

2.





3.



Fiecare pull request trebuie să fie făcut de către un contributor diferit.

Context Proiect

în Uniq Contributor: self. requests → isUniq (comit)

Model subiect 2026

1. Dezvoltarea modelelor în procesul de dezvoltare a software-ului este esențială pentru:

- înțelegerea corectă a cerințelor și eliminarea ambiguităților
- determinarea erorilor de proiectare timpuriu, acest lucru reducând semnificativ costurile unor intervenții ulterioare (dacă aceste erori se descopereau în faza de testare sau chiar după punerea în funcțiune a sistemului)
- obținerea unei soluții mai bune din punct de vedere al mentenanței și extensibilității pe viitor
- clarificarea strategiei atât între programatori, dezvoltatori, cât și clienți

2. În ambele situații a căsa continuă a instanțelor a altei clase, în să existe corectările diferențe;

→ agregare:

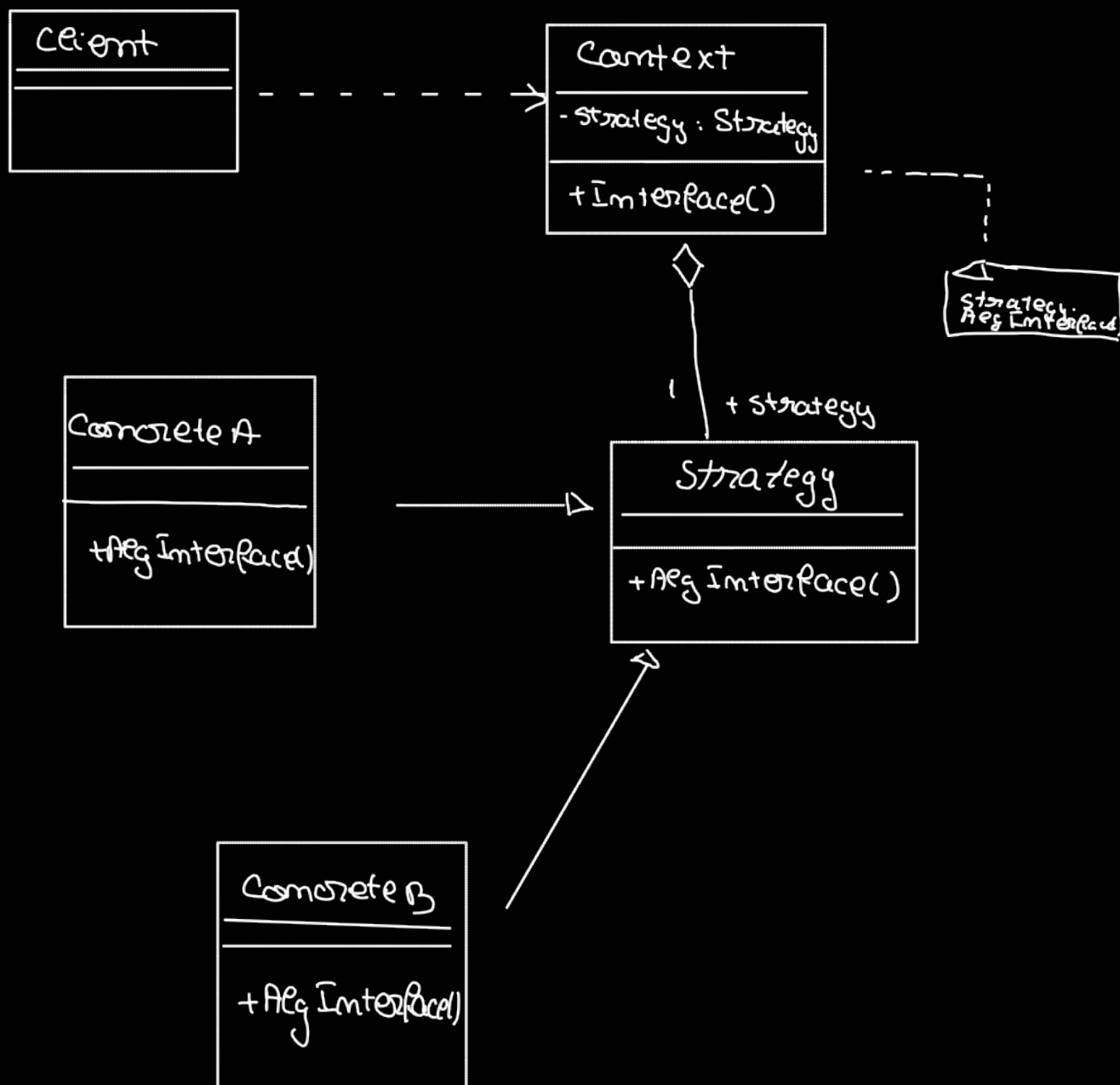
- o obiectul inclus poate exista în absența obiectului din care face parte (ex: avem o cutie cu carne, dar carnea poate exista fără cutie; este un obiect de sine stătător)
- o partea poate aparține mai multor întregi simultan

→ compunere:

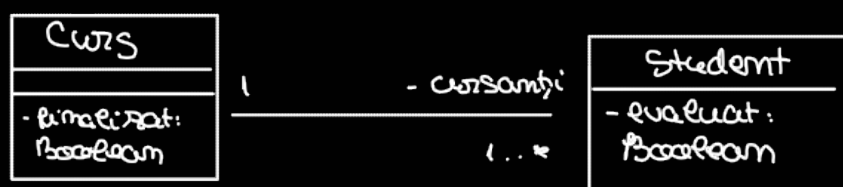
- o partea nu poate exista fără întreg; nu este un obiect de sine stătător (ex: cardurile, camere; a cardului are camere, dar o cameră nu poate exista în absența unei carduri)
- o partea aparține UNUI SINGUR întreg

3. Sistemul de proiectare Strategy.

Acest sistem de proiectare permite utilizarea interschim. Căutarea a mai multor metode de rezolvare, prin utilizarea unei clase abstracte Strategy și a unor subclase concrete ce implementează metodele de rezolvare. **POT FI ADĂUGAȚI ALG. NOI FĂRĂ A MODIFICA CONTEXTUL SAU CLIENTUL.**



4. invariant care să reflecteze iteratorul pe care îl are pe obiect



Se ia ca exemplu situatia in care Studentii se pot inscrie la un singur curs pe timpul verii (ex: Educatie Ambientala). Cursul poate avea mai multi studenti, iar acestia, initial, nu sunt evaluati.

constrangeri : Ca un curs sa se considere finalizat, toti studentii trebuie sa fie evaluati.

context Curs

```
inv Setare Finalizat : self.finalizat = true implies  
    self.cursanti →  
    free (evaluat)
```